

Chương này trình bày các khái niệm lý thuyết và công nghệ nền tảng làm cơ sở cho toàn bộ hệ thống được xây dựng trong đề án. Mỗi mục tập trung vào nguyên lý hoạt động và lý do lựa chọn công nghệ, không đi vào chi tiết cài đặt cụ thể; phần đó được trình bày trong Chương ?? (Thiết kế) và Chương ?? (Cài đặt).

0.1 Blockchain và Ethereum

0.1.1 Mô hình blockchain và cơ chế đồng thuận

Blockchain là một cấu trúc dữ liệu phân tán, trong đó dữ liệu được tổ chức thành các khối (block) liên kết với nhau theo chuỗi thông qua hàm băm mật mã. Mỗi block chứa tập hợp các giao dịch (transaction), hash của block trước đó, và một số định danh duy nhất. Cấu trúc này đảm bảo tính bất biến: thay đổi bất kỳ block nào sẽ làm thay đổi hash của toàn bộ chuỗi phía sau, và mạng lưới sẽ từ chối nhánh bị thay đổi đó [1], [2].

Ethereum ra mắt năm 2015, là blockchain thế hệ hai với khả năng thực thi hợp đồng thông minh, cho phép lưu trữ logic nghiệp vụ tùy ý trên chuỗi, không chỉ giới hạn ở giao dịch chuyển token như Bitcoin. Từ tháng 9/2022, sau sự kiện "The Merge", Ethereum chuyển hoàn toàn sang cơ chế đồng thuận Proof of Stake (PoS), thay thế Proof of Work (PoW) trước đó [2].

Trong PoS, mạng lưới được bảo vệ bởi các validator, tức là những node đặt cọc (stake) tối thiểu 32 ETH để tham gia xác nhận giao dịch. Thời gian trên PoS Ethereum được tổ chức thành hai đơn vị: (i) Slot (12 giây): Tại mỗi slot, một validator được chọn ngẫu nhiên bởi giao thức để đề xuất (propose) một block mới. Đồng thời, một ủy ban (committee) gồm các validator khác được chỉ định để bỏ phiếu xác nhận (attestation) tính hợp lệ của block đó. Nếu validator được chọn không đề xuất block kịp thời (do offline hoặc lỗi mạng), slot đó bị bỏ trống (missed slot). (ii) Epoch (32 slot = 6,4 phút): Đơn vị thời gian để đánh giá và xác nhận trạng thái chuỗi. Sau mỗi epoch, các block được xác nhận sơ bộ thông qua cơ chế bỏ phiếu của toàn bộ tập validator. Khi 2 epoch liên tiếp đều được justified, epoch đầu tiên trở thành xác nhận vĩnh viễn; các block trong epoch đó không thể bị thay đổi trừ khi ít nhất 1/3 tổng ETH đặt cọc bị tiêu hủy (slashed). Cơ chế PoS có hai ưu điểm quan trọng so với PoW: giảm tiêu thụ năng lượng hơn 99%, và cung cấp cơ chế finality xác định (deterministic finality), nghĩa là sau khi finalize, block là bất biến, khác với PoW nơi xác nhận chỉ mang tính xác suất.

0.1.2 Ethereum Virtual Machine (EVM) và Gas

EVM là máy tính ảo thực thi smart contract trên Ethereum. Mỗi node trong mạng chạy một bản sao EVM và thực thi các lệnh theo cùng thứ tự, đảm bảo kết quả đồng nhất trên toàn mạng; đây là nền tảng cho tính chất deterministic của smart

contract [2].

Gas là đơn vị đo lường chi phí tính toán trên EVM. Mỗi lệnh (opcode) có chi phí gas cố định — ví dụ phép cộng (ADD) tốn 3 gas, trong khi ghi vào storage (SSTORE) tốn khoảng 20.000 gas. Người gửi transaction phải trả gas fee bằng ETH; nếu gas hết trước khi transaction hoàn thành, toàn bộ thay đổi được hoàn tác (revert) nhưng gas đã dùng vẫn bị tính. Cơ chế này ngăn chặn các vòng lặp vô hạn và tấn công từ chối dịch vụ (DoS). Chi phí SSTORE cao nhất trong các opcode chính là lý do cốt lõi tại sao thiết kế smart contract phải tránh cập nhật storage cho người dùng trong một transaction; bài toán này được giải quyết bởi cơ chế Global Interest Index trình bày tại mục ??.

Kể từ bản nâng cấp London (tháng 8/2021), Ethereum áp dụng cơ chế phí gas theo chuẩn EIP-1559. Phí gas được chia thành hai thành phần: base fee — mức phí tối thiểu do giao thức tự động điều chỉnh theo mức độ sử dụng block, bị đốt hoàn toàn và không trả cho validator; và priority fee (tip) — khoản phí tùy chọn trả cho validator để giao dịch được ưu tiên. Tổng phí được tính bằng công thức

$$\text{Gas fee} = \text{Gas used} \times (\text{Base fee} + \text{Priority fee}) \quad (1)$$

Cơ chế này giúp phí gas trở nên minh bạch và dễ dự đoán hơn so với mô hình đấu giá trước đó [2].

Đối với đề án, EIP-1559 không ảnh hưởng đến logic nghiệp vụ bên trong smart contract, nhưng ảnh hưởng trực tiếp đến chi phí mỗi lần người dùng tương tác với giao thức (deposit, borrow, repay, withdraw). Vì vậy, tối ưu gas trong thiết kế smart contract, chẳng hạn như giảm số lượng SSTORE và sử dụng Lazy Accrual thay vì Eager Accrual, có ý nghĩa thực tế quan trọng.

0.1.3 Smart contract và tính bất biến

Smart contract là chương trình được lưu trữ trên blockchain và thực thi tự động trong transaction. Khi đã deploy, bytecode của smart contract không thể thay đổi; đây là tính chất bất biến (immutability) của blockchain. Tính chất này đảm bảo người dùng có thể kiểm chứng logic của smart contract và tin rằng nó sẽ hoạt động đúng như cam kết.

Tuy nhiên, tính bất biến cũng tạo ra thách thức: khi phát hiện lỗ hổng bảo mật hoặc cần nâng cấp tính năng, không thể sửa smart contract đã deploy. Để giải quyết, hệ thống sử dụng Proxy Pattern — một kỹ thuật tách biệt logic khỏi dữ liệu, cho phép thay thế phần logic trong khi giữ nguyên địa chỉ smart contract và dữ liệu người dùng. Chi tiết về UUPS Proxy Pattern được trình bày tại mục 0.4.

0.1.4 Chuẩn token ERC-20

ERC-20 (Ethereum Request for Comment 20) là chuẩn giao diện (interface) phổ biến nhất cho token trên Ethereum, được đề xuất bởi Fabian Vogelsteller và Vitalik Buterin vào năm 2015. Chuẩn này quy định một tập hợp tối thiểu các hàm mà mọi token contract phải triển khai, bao gồm: `transfer()` để chuyển token giữa các địa chỉ, `approve()` và `transferFrom()` để cho phép một địa chỉ khác (thường là smart contract) chi tiêu token thay mặt chủ sở hữu, cùng với `balanceOf()` để truy vấn số dư và `totalSupply()` để biết tổng cung [2].

Chuẩn ERC-20 là nền tảng kỹ thuật trực tiếp của đồ án: toàn bộ tài sản mà giao thức hỗ trợ đều là token ERC-20. Khi người dùng gửi tài sản vào LendingPool, quy trình bắt buộc gồm hai bước: (1) người dùng gọi `approve(lendingPoolAddress, amount)` trên token contract để cấp quyền, (2) LendingPool gọi `transferFrom(user, address(this), amount)` để chuyển token vào pool. Cơ chế approve-transferFrom này đảm bảo chỉ khi người dùng chủ động cấp quyền, smart contract mới có thể di chuyển token, từ đó ngăn chặn truy cập trái phép.

Một vấn đề thực tế quan trọng là không phải mọi token ERC-20 đều tuân thủ chuẩn một cách đầy đủ: một số token không trả về giá trị `bool` khi gọi `transfer()`, một số khác trả về `false` thay vì `revert` khi thất bại. Để xử lý các trường hợp bất thường này, LendingPool sử dụng thư viện `SafeERC20` của OpenZeppelin — bọc lại các lời gọi `transfer` và `revert` nếu token báo lỗi, đảm bảo an toàn cho mọi loại ERC-20 (xem tầng 1 bảo mật tại mục ??) [3].

Ngoài ra, mỗi token ERC-20 có thuộc tính `decimals` xác định số chữ số thập phân: ví dụ WETH có 18 decimals trong khi USDC chỉ có 6 decimals. Sự khác biệt này đòi hỏi giao thức phải chuẩn hóa về cùng một không gian (18 decimals) khi tính toán giá trị USD, đặc biệt trong quá trình thanh lý (xem mục ??).

0.2 DeFi Lending — Mô hình và cơ chế kinh tế

0.2.1 Mô hình pool thanh khoản chung

Tài chính phi tập trung (Decentralized Finance, viết tắt là DeFi) là hệ sinh thái ứng dụng tài chính vận hành trên blockchain, không cần trung gian tập trung. Trong mô hình lending phi tập trung, người gửi tiền (depositor) và người vay (borrower) không giao dịch trực tiếp với nhau mà thông qua một pool thanh khoản chung được quản lý bởi smart contract [4].

Cách hoạt động của pool: người gửi đưa tài sản vào pool và nhận lãi suất từ người vay; người vay rút tài sản từ pool với điều kiện thế chấp tài sản khác có giá trị cao hơn khoản vay. Smart contract tự động tính toán lãi suất, theo dõi số dư và

thực thi thanh lý khi cần; vì thế, không cần nhân viên ngân hàng, hợp đồng giấy hay thủ tục phê duyệt.

Ưu điểm của mô hình pool so với mô hình peer-to-peer truyền thống: người gửi không cần chờ được khớp với người vay cụ thể; người vay có thể vay ngay lập tức nếu pool có đủ thanh khoản; lãi suất được điều chỉnh tự động theo cung cầu thị trường.

0.2.2 Overcollateralization và lý do tại sao

Khác với ngân hàng truyền thống có thể cho vay tín chấp (dựa trên lịch sử tín dụng và thu nhập), DeFi lending yêu cầu người vay thế chấp tài sản có giá trị lớn hơn khoản vay — gọi là overcollateralization (thế chấp quá mức) [4].

Lý do bắt buộc phải overcollateralize: trên blockchain, không có cơ chế pháp lý để truy đòi con nợ như trong thế giới thực. Nếu người vay không trả nợ và biến mất, giao thức không thể kiện hay thu hồi tài sản. Do đó, giao thức phải nắm giữ tài sản thế chấp có giá trị đủ lớn để tự bảo vệ — ngay cả khi giá tài sản thế chấp giảm một phần, giao thức vẫn có thể bán ra để thu hồi khoản nợ.

Collateral Factor (hệ số thế chấp) là tỷ lệ tối đa mà người vay có thể vay so với giá trị tài sản thế chấp. Ví dụ, collateral factor = 0,75 nghĩa là với 100 USD tài sản thế chấp, tối đa có thể vay 75 USD. Hệ số này có thể được đặt khác nhau cho từng tài sản dựa trên mức độ rủi ro: tài sản ổn định như USDC có thể có collateral factor cao hơn tài sản biến động như các token nhỏ.

0.2.3 Health Factor và cơ chế thanh lý

Health Factor (hệ số sức khỏe) là chỉ số đo lường mức độ an toàn của một vị thế vay. Công thức tổng quát:

$$\text{Health Factor} = \frac{\text{Tổng giá trị tài sản thế chấp} \times \text{liquidationThreshold}}{\text{Tổng giá trị nợ}} \quad (2)$$

Khi Health Factor ≥ 1 , vị thế an toàn. Khi Health Factor < 1 , vị thế có thể bị thanh lý — người thanh lý (liquidator) sẽ trả một phần nợ thay cho người vay và nhận lại tài sản thế chấp tương ứng với giá chiết khấu (liquidation incentive). Cơ chế này tạo ra động lực kinh tế để bên thứ ba chủ động duy trì sức khỏe của giao thức.

Liquidation Threshold khác với Collateral Factor: Collateral Factor xác định mức tối đa có thể vay ban đầu trong khi Liquidation Threshold xác định ngưỡng mà tại đó vị thế sẽ bị thanh lý. Thông thường Liquidation Threshold sẽ lớn hơn

Collateral Factor, tạo ra một vùng đệm an toàn để người vay có thời gian phản ứng khi giá tài sản giảm.

0.2.4 Utilization Rate và cơ chế điều chỉnh lãi suất

Utilization Rate (tỷ lệ sử dụng vốn) $U = \text{TotalBorrows} / \text{TotalDeposits}$ là chỉ số trung tâm điều phối cân bằng cung cầu trong giao thức.

Khi U cao (gần 100%), pool sắp cạn kiệt thanh khoản: người gửi không thể rút tiền, rủi ro hệ thống tăng. Lúc này, lãi suất vay và gửi phải tăng cao để: thu hút thêm người gửi tiền và buộc người vay trả nợ sớm từ đó tái cân bằng pool. Khi U thấp, tiền trong pool nhàn rỗi, lãi suất giảm để khuyến khích vay nhiều hơn, tối đa hóa lợi suất cho giao thức.

Mối liên hệ giữa utilization rate và lãi suất là nền tảng lý thuyết của mô hình Two-Slope được trình bày chi tiết ở mục 0.3 bên dưới.

0.3 Mô hình lãi suất Two-Slope (Kinked Rate Model)

0.3.1 Lý thuyết kinh tế của mô hình

Mô hình Two-Slope, còn gọi là Kinked Rate Model, là mô hình lãi suất chuẩn được sử dụng trong hầu hết các giao thức DeFi lending lớn (Aave, Compound) [5], [6]. Ý tưởng cốt lõi: lãi suất không tăng đều tuyến tính theo utilization rate, mà tăng theo hai giai đoạn với tốc độ khác nhau, tạo thành dạng đường gãy (kinked).

Giai đoạn 1 — Vùng bình thường ($U \leq U_{\text{optimal}}$): Lãi suất tăng chậm và tuyến tính. Mục tiêu là duy trì chi phí vay hợp lý để khuyến khích sử dụng vốn hiệu quả, trong khi vẫn đủ hấp dẫn để thu hút người gửi.

Giai đoạn 2 — Vùng quá tải ($U > U_{\text{optimal}}$): Lãi suất tăng rất nhanh với hệ số dốc lớn hơn nhiều ($\text{slope2} \gg \text{slope1}$). Tại đây, giao thức đang tiến đến nguy cơ mất cân bằng thanh khoản, lãi suất cao đột ngột tạo áp lực kinh tế cấp bách buộc người vay trả nợ, đồng thời thu hút dòng tiền mới từ người gửi.

U_{optimal} (thường 70-80%) là điểm cân bằng tối ưu về lý thuyết: pool sử dụng vốn hiệu quả nhưng vẫn giữ đủ thanh khoản cho người gửi muốn rút. Đây là tham số quan trọng nhất trong việc hiệu chỉnh hành vi kinh tế của giao thức.

0.3.2 Reserve Factor — Cơ chế doanh thu giao thức

Reserve Factor là tỷ lệ phần trăm lãi từ người vay được giữ lại cho ngân quỹ giao thức, thay vì trả toàn bộ cho người gửi. Về mặt kinh tế, đây là cơ chế tạo doanh thu bền vững: giao thức tích lũy nguồn dự phòng từ hoạt động thực tế, không phụ thuộc vào token inflation hay trợ cấp bên ngoài. Nguồn này có thể dùng để bù đắp tổn thất khi thanh lý không kịp, trả tiền cho nhóm phát triển, hoặc phát triển hệ sinh

thái.

0.4 UUPS Upgradeable Proxy Pattern

0.4.1 Vấn đề và nhu cầu proxy

Như đã đề cập ở mục 0.1, smart contract sau khi deploy là bất biến. Với một giao thức tài chính thực tế, điều này tạo ra rủi ro nghiêm trọng: nếu phát hiện lỗi hỏng bảo mật, không thể vá mà không mất toàn bộ dữ liệu người dùng và TVL. Người dùng cũng sẽ phải migrate tài sản sang smart contract mới — một quy trình phức tạp, tốn kém và rủi ro.

Proxy Pattern giải quyết vấn đề này bằng cách tách một smart contract thành hai phần: Proxy Contract lưu toàn bộ state (số dư người dùng, tham số giao thức,...) tại địa chỉ cố định; Implementation Contract chứa logic nghiệp vụ tại địa chỉ có thể thay thế. Người dùng luôn tương tác với địa chỉ proxy. Khi cần nâng cấp, chỉ deploy implementation mới và trỏ proxy sang đó, từ đó state được giữ nguyên mà logic thực thi trên state đó thì thay đổi [3], [7].

0.4.2 Cơ chế delegatecall

Delegatecall là opcode EVM cho phép smart contract A gọi code của smart contract B nhưng thực thi trong context (storage, msg.sender, msg.value) của smart contract A. Đây là nền tảng của mọi proxy pattern: khi người dùng gọi hàm deposit() trên proxy, proxy dùng delegatecall để chạy code deposit() từ implementation, nhưng đọc và ghi vào storage của chính proxy — không phải của implementation. Người dùng nhìn thấy proxy như một smart contract đơn lẻ bình thường.

0.4.3 So sánh ba loại proxy phổ biến

Tiêu chí	Transparent Proxy	UUPS	Beacon Proxy
Logic upgrade	Trong Proxy Contract	Trong Implementation Contract	Trong Beacon Contract
Chi phí gas mỗi call	Cao hơn (kiểm tra admin)	Thấp nhất	Trung bình
Chi phí upgrade	Trung bình	Cao hơn một chút (deploy impl mới)	Thấp (1 thay đổi, nhiều proxy cập nhật)
Độ phức tạp	Thấp	Trung bình	Cao
Phù hợp khi	Contract đơn lẻ, upgrade thường xuyên	Contract cốt lõi, tối ưu gas	Nhiều proxy cùng implementation

Bảng 0.1: So sánh 3 loại proxy phổ biến

Sau khi so sánh cùng với các giải pháp thay thế trong bảng 0.1, UUPS được lựa chọn cho giao thức vì hai lý do chính. Thứ nhất, chi phí gas mỗi lần call thấp nhất trong ba loại proxy do không có logic kiểm tra admin tại mỗi lần dispatch. Thứ hai, logic upgrade nằm trong implementation contract, cho phép kiểm soát điều kiện nâng cấp ngay tại tầng logic; ví dụ, chỉ controller mới có quyền upgrade, thay vì phải lưu logic phân quyền riêng trong proxy. Tuy nhiên, thiết kế này đi kèm một rủi ro đặc thù cần lưu ý: nếu implementation mới được deploy thiếu hàm `_authorizeUpgrade`, proxy sẽ mất khả năng upgrade vĩnh viễn vì không còn entry point nào để thực hiện. Đây là điểm khác biệt so với Transparent Proxy, nơi logic upgrade nằm trong proxy nên không bị ảnh hưởng bởi nội dung của implementation. Rủi ro này được kiểm soát trong đề án bằng hai cơ chế: toàn bộ test case upgrade trong `ProtocolController.test.ts` đều xác minh hàm `_authorizeUpgrade` tồn tại và hoạt động đúng trên implementation mới; và việc upgrade bắt buộc phải đi qua `ProtocolController` với `timelock delay`, tạo cửa sổ thời gian để phát hiện lỗi trước khi lệnh được thực thi.

0.4.4 Storage Layout và rủi ro xung đột slot

Một vấn đề đáng chú ý trong UUPS: khi upgrade implementation mới, thứ tự khai báo các biến state phải giữ nguyên hoàn toàn so với implementation cũ. Nếu thêm biến mới vào giữa hoặc xóa biến cũ, các biến tiếp theo sẽ bị dịch chuyển sang slot storage khác, dẫn đến đọc nhầm dữ liệu — một loại lỗi cực kỳ nguy hiểm vì không gây revert mà âm thầm trả về giá trị sai.

Giải pháp chuẩn là khai báo một mảng `__gap[ ]` ở cuối storage contract, dự trữ một số lượng slot nhất định cho các biến tương lai. Khi cần thêm biến mới, chỉ cần thu nhỏ `__gap[ ]` tương ứng, khi đó thứ tự các biến hiện tại không bị thay đổi.

0.5 Oracle và vấn đề giá tài sản on-chain

0.5.1 Oracle Problem trong blockchain

Smart contract hoạt động trong một môi trường khép kín: EVM chỉ có thể đọc dữ liệu đã tồn tại trên blockchain, không thể tự lấy dữ liệu từ thế giới bên ngoài (giá cổ phiếu, tỷ giá, kết quả thể thao...). Đây là Oracle Problem, tức là một thách thức cơ bản của blockchain.

Với DeFi lending, việc tính health factor và giá trị tài sản thế chấp đòi hỏi phải biết giá thị trường của từng tài sản tại thời điểm hiện tại. Nếu giá nguồn cấp không chính xác hoặc bị thao túng, kẻ tấn công có thể thao túng thanh lý để thu lợi bất chính, hoặc vay nhiều hơn mức được phép. Do đó, nguồn cấp giá là một trong những điểm tấn công quan trọng nhất của bất kỳ lending protocol nào.

0.5.2 Chainlink Data Feed — Giải pháp oracle phi tập trung

Chainlink là mạng lưới oracle phi tập trung lớn nhất, cung cấp dữ liệu giá cho hầu hết các giao thức DeFi lớn bao gồm Aave và Compound. Chainlink Data Feed hoạt động theo mô hình aggregation: nhiều node độc lập lấy giá từ nhiều sàn giao dịch khác nhau và báo cáo vào một Aggregator contract trên chain. Giá cuối cùng là tổng hợp của tất cả báo cáo, loại bỏ ảnh hưởng của nguồn đơn lẻ bị lỗi hoặc bị tấn công [8].

Chainlink Data Feed có hai cơ chế cập nhật: Heartbeat — cập nhật định kỳ sau một khoảng thời gian nhất định (ví dụ 24 giờ) ngay cả khi giá không thay đổi nhiều; và Deviation Threshold — cập nhật ngay khi giá thay đổi quá một ngưỡng nhất định (ví dụ 0,5%). Kết hợp hai cơ chế này đảm bảo giá on-chain luôn gần với giá thực tế thị trường.

0.5.3 Stale Price — Rủi ro giá cũ

Stale Price xảy ra khi Chainlink node ngừng cập nhật, có thể là do sự cố mạng, lỗi node, hoặc thị trường bị gián đoạn. Smart contract không có cơ chế tự động phát hiện điều này; nó chỉ đọc giá mới nhất trên chain, có thể là giá đã lỗi thời hàng giờ đồng hồ. Rủi ro: nếu giá thực tế đã giảm mạnh nhưng oracle vẫn báo giá cao, người dùng có thể vay vượt mức an toàn thực tế. Ngược lại, giá thực tế tăng nhưng oracle vẫn báo giá thấp, người dùng có thể bị thanh lý oan.

0.6 Message Queue và kiến trúc Event-Driven

0.6.1 Message Broker và Producer-Consumer Pattern

Message broker là thành phần trung gian nhận message từ producer (bên gửi) và chuyển đến consumer (bên nhận). Khác với gọi hàm trực tiếp, message broker tách biệt hoàn toàn producer và consumer về mặt thời gian và không gian: producer gửi message vào queue và tiếp tục công việc ngay lập tức mà không cần chờ consumer xử lý xong; consumer xử lý message theo tốc độ của mình, có thể chậm hơn producer [9].

Kiến trúc Event-Driven xây dựng hệ thống xung quanh luồng sự kiện: mỗi thay đổi trạng thái quan trọng được biểu diễn dưới dạng event và truyền đi qua message broker. Các thành phần quan tâm đến event đó sẽ subscribe và xử lý độc lập. Kiến trúc này phù hợp tự nhiên với blockchain, nơi mọi thay đổi trạng thái đều được phát ra dưới dạng event (Deposit, Borrow, Repay, ...).

0.6.2 Các đảm bảo delivery và trade-off

Mức đảm bảo	Ý nghĩa	Rủi ro	Phù hợp với
At-most-once	Message được xử lý tối đa 1 lần	Mất message nếu consumer crash	Log không quan trọng, metrics
At-least-once	Message được xử lý ít nhất 1 lần	Duplicate nếu crash sau xử lý nhưng trước ack	Dữ liệu tài chính quan trọng (với idempotency)
Exactly-once	Message được xử lý đúng 1 lần	Chi phí cao, phức tạp, throughput thấp	Giao dịch ngân hàng nghiêm ngặt

Bảng 0.2: So sánh các đảm bảo delivery

Bảng 0.2 đã đặt lên bàn cân 3 giải pháp delivery, trong đó, At-least-once được lựa chọn cho hệ thống backend của đồ án vì mất event blockchain là không thể chấp nhận (dữ liệu sẽ lệch với on-chain), trong khi duplicate có thể xử lý bằng idempotency ở tầng ứng dụng với chi phí thấp hơn nhiều so với exactly-once.

0.6.3 Idempotency — Xử lý hệ quả của at-least-once

Idempotency là tính chất của một thao tác mà thực hiện nhiều lần cho ra kết quả giống hệt thực hiện một lần. Với at-least-once delivery, một event có thể được xử lý nhiều lần (ví dụ: consumer crash sau khi lưu database nhưng trước khi ack). Nếu handler không idempotent, database sẽ có bản ghi trùng lặp.

Cách thiết kế idempotent handler: sử dụng unique constraint trên trường định danh tự nhiên của event (ví dụ: transactionHash trên Ethereum là duy nhất toàn cầu). Khi event được xử lý lần thứ hai, lệnh INSERT vi phạm constraint và bị bỏ qua — thay vì tạo bản ghi trùng hay ném lỗi.

0.7 Sign-In with Ethereum (EIP-4361)

0.7.1 Vấn đề xác thực trong ứng dụng phi tập trung

Ứng dụng Web2 truyền thống xác thực người dùng bằng username/password hoặc OAuth với bên thứ ba (Google, Facebook). Cả hai cách đều giả định tồn tại một cơ sở dữ liệu tập trung lưu thông tin tài khoản — mâu thuẫn với triết lý phi tập trung của DeFi.

Trong Web3, mỗi người dùng đã có một danh tính duy nhất: địa chỉ ví Ethereum, được tạo ra từ cặp public/private key. Nếu người dùng có thể chứng minh họ sở hữu private key của một địa chỉ mà không lộ key đó, đây chính là cơ chế xác thực hoàn hảo: không cần đăng ký, không có password để bị lộ, không phụ thuộc bên thứ ba.

0.7.2 Chuẩn EIP-4361 và cơ chế hoạt động

EIP-4361 (Sign-In with Ethereum) chuẩn hóa quy trình xác thực bằng ví Ethereum. Quy trình cốt lõi: backend tạo một message plaintext có cấu trúc cố định (địa chỉ ví, nonce ngẫu nhiên, chainId, thời gian hết hạn); người dùng ký message này bằng private key của ví; backend xác minh chữ ký bằng địa chỉ ví tương ứng [10].

Tính bảo mật đến từ hai yếu tố: (i) chữ ký ECDSA là bằng chứng toán học về việc sở hữu private key, không thể giả mạo; (ii) nonce ngẫu nhiên trong message ngăn replay attack, vì mỗi session có một nonce khác nhau nên chữ ký cũ không thể dùng lại.

Sơ đồ tuần tự chi tiết của luồng xác thực được trình bày bằng hình ??, cài đặt cụ thể trong hệ thống được nói đến tại mục ??.

0.8 Lý do lựa chọn công nghệ

Bảng 0.3 dưới đây tổng hợp lý do lựa chọn công nghệ cụ thể cho từng tầng, bổ sung cho định hướng kiến trúc tổng thể đã trình bày ở mục ??. Các lựa chọn được tham khảo từ tài liệu chính thức của các nền tảng và thư viện liên quan [3], [9], [11], [12].

Tầng	Công nghệ được chọn	Lý do chính	Lựa chọn thay thế đã xem xét
Smart contract	Solidity + Hardhat	Ngôn ngữ chuẩn cho Ethereum, hệ sinh thái phong phú, OpenZeppelin library	Vyper (ít tài liệu hơn), Foundry (tốt cho testing nhưng Hardhat linh hoạt hơn cho deploy)
Smart contract	UUPS Proxy	Gas thấp nhất cho mỗi call, logic upgrade trong implementation tăng bảo mật	Transparent Proxy (gas cao hơn), Beacon Proxy (phức tạp không cần thiết)
Backend	Node.js + TypeScript	Cùng hệ sinh thái với ethers.js, async/event-driven phù hợp với blockchain	Python (ít library blockchain hơn), Go (mạnh hơn nhưng ít library DeFi)
Backend	RabbitMQ	Đủ throughput cho quy mô đồ án, cài đặt đơn giản, hỗ trợ tốt acknowledgment	Kafka (overkill cho quy mô này), Redis Streams (thiếu một số tính năng)
Backend	PostgreSQL	Hỗ trợ DECIMAL(78,0) cho uint256, mature và đáng tin cậy	MySQL (tương tự), MongoDB (schema linh hoạt hơn nhưng kém transaction)
Frontend	Next.js + ethers.js v6	SSR giúp tải trang nhanh, ethers.js là library chuẩn cho Ethereum Web3	Vite/React thuần (không có SSR), web3.js (ít được maintain hơn ethers.js)

Bảng 0.3: Các công nghệ được sử dụng